

TP SERVEUR UNIX L3

RANDRIAMANDROSOMANANA Mihaja Helisoa Info L3

AMBOARANIRINA FANOMEZANJANAHARY Koloina Sarobidy Info L3

Partie 0: Organisation des fichiers:

 **Serveur-C** Public

 Pin  Watch **0**  Fork **0**  Star **0**

 3ab7a8f   Go to file Code

 **Mihaja07** Partie 1 — Serveur TCP itératif 3ab7a8f · 7 hours ago 

 include	Partie 0: Organisation des fic...	7 hours ago
 src	Partie 1 — Serveur TCP itératif	7 hours ago
 Makefile	Partie 0: Organisation des fic...	7 hours ago
 README.md	Initial commit	7 hours ago
 rapport.docx	Partie 0: Organisation des fic...	7 hours ago
 syslog.conf	Partie 0: Organisation des fic...	7 hours ago

About

TP serveur UNIX L3

-  Readme
-  Activity
-  **0** stars
-  **0** watching
-  **0** forks

Releases

No releases published

[Create a new release](#)

Partie 1: Serveur TCP iteratif

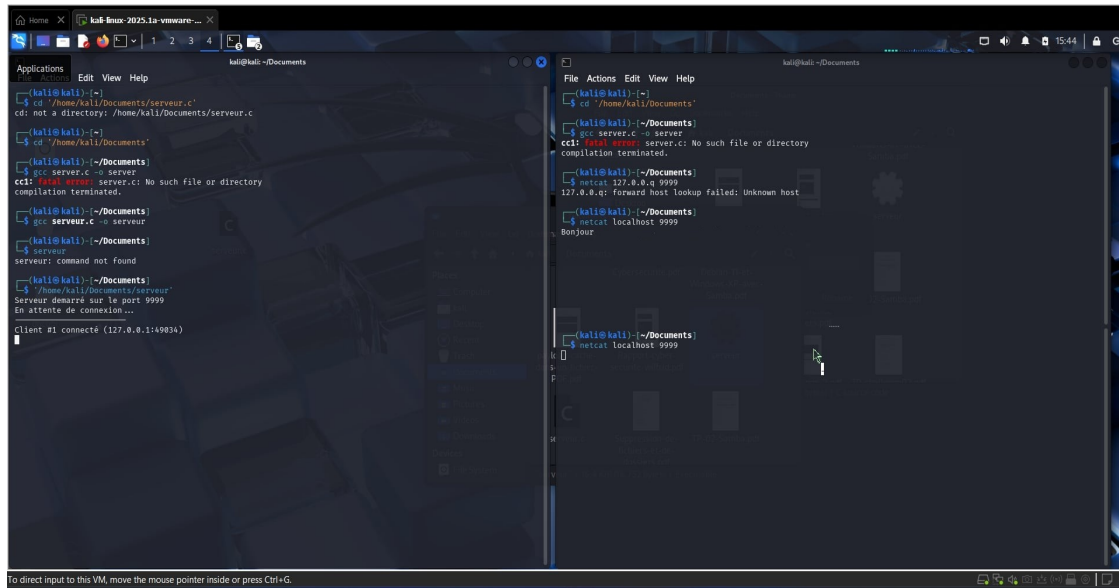
Le serveur tcp attend qu'un client se connecte et attend que ce client envoie un message, le serveur lit ce message et le renvoie au client en ajoutant "[Connexion #N] Echo : <message>"

Les erreurs read et write de socket sont afficher avec perror, puis le programme continue de fonctionner, les clients sont traiter un par un, les autre connection doivent



Edit with WPS Office

attendre, le serveur fini de traiter le client actuel avant de passer en mode accept()



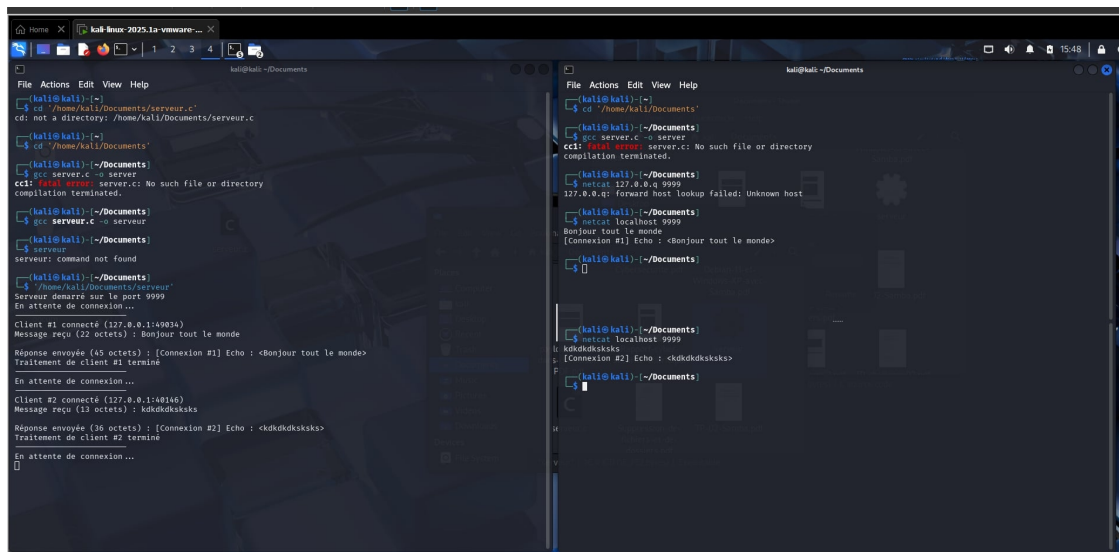
```
kali@kali:~/Documents$ cd /home/kali/Documents/serveur.c
cd: not a directory: /home/kali/Documents/serveur.c

(kali@kali)~$ cd /home/kali/Documents
(kali@kali)~/Documents$ gcc server.c -o server
cc1: fatal error: server.c: No such file or directory
compilation terminated.

(kali@kali)~/Documents$ gcc server.c -o serveur
(kali@kali)~/Documents$ ./serveur
Serveur démarré sur le port 9999
En attente de connexion...

client #1 connecté (127.0.0.1:49034)
```

Comme on le voit dans l'image, seul le client #1 est connecté, car le serveur doit traiter la connexion #1 avant de revenir a accept()



```
(kali@kali)~/Documents$ gcc server.c -o server
cc1: fatal error: server.c: No such file or directory
compilation terminated.

(kali@kali)~/Documents$ gcc server.c -o serveur
(kali@kali)~/Documents$ ./serveur
Serveur démarré sur le port 9999
En attente de connexion...

Client #1 connecté (127.0.0.1:49034)
Message reçu (22 octets) : Bonjour tout le monde
Réponse envoyée (45 octets) : [Connexion #1] Echo : <Bonjour tout le monde>
Traitement de client #1 terminé

En attente de connexion...

Client #2 connecté (127.0.0.1:49146)
Message reçu (13 octets) : kdkdkdkskks
Réponse envoyée (36 octets) : [Connexion #2] Echo : <kdkdkdkskks>
Traitement de client #2 terminé

En attente de connexion...
```

Dans le terminal, on voit clairement que le traitement du client #1 est terminé puis le serveur revient a accept() et accepte le client #2

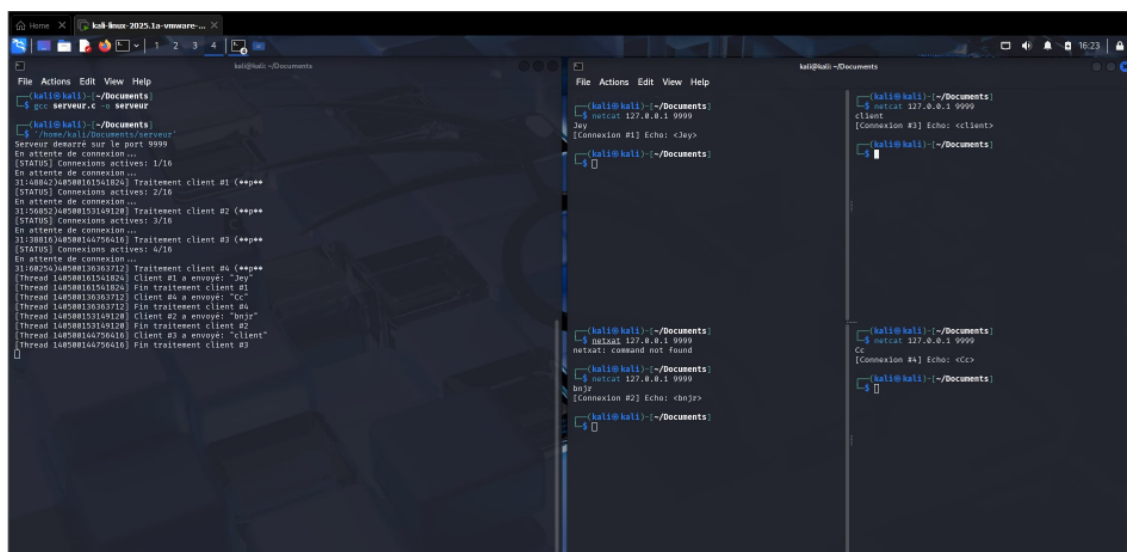
Partie 2: Serveur concurrent avec fork()

Le serveur est maintenant concurrent en utilisant fork(), apres que le serveur accepte une connexion, un processus fils traite cette connexion et le père retourne a accept()

Pour compter le nombre de connexion active dans le serveur, on a créé un fichier temporaire pour stocker cette valeur, c'est plus facile a mettre en place et facile a comprendre, quand un processus fils meurt, il soustrait cette valeur de 1 et met le fichier a jour

sigchld_handler() est un gestionnaire de signal qui nettoie automatiquement les processus fils terminés pour éviter la création de processus zombies

Partie 3: Version multi-threadée avec pthreads



```
kal@kali:~/Documents$ ./serveur
Serveur demarre sur le port 9999
En attente de connexion...
[STATUS] Connexions actives: 1/16
En attente de connexion...
31:0862340808181541824] Traitement client #1 (***)
[STATUS] Connexions actives: 2/16
En attente de connexion...
31:50852340808153149128] Traitement client #2 (***)
[STATUS] Connexions actives: 2/16
En attente de connexion...
31:38010340808144756416] Traitement client #3 (***)
[STATUS] Connexions actives: 4/16
En attente de connexion...
31:082340808130383722] Traitement client #4 (***)
[Thread 140808151541824] Fin traitement client #1
[Thread 140808130383722] Client #4 a envoye: 'Cc'
[Thread 140808130383722] Fin traitement client #4
[Thread 14080813148128] Client #2 a envoye: 'bnjr'
[Thread 140808153149128] Fin traitement client #2
[Thread 140808144756416] Client #3 a envoye: 'client'
[Thread 140808144756416] Fin traitement client #3
```

Le serveur utilise des threads, different processus peuvent donc avoir accès a une variable globale, et c'est ce qu'on a utiliser ici pour compter le nombre de connexion active dans le serveur au lieu de le stocker dans un fichier, ici, 4 clients se connectent simultanément et le serveur les gère tres bien



Partie: 4 Multiplexage I/O avec select() et poll()

Dans cette partie, nous avons implémenté un serveur totalement différent des précédents. Fini les processus fils avec fork() et les threads avec pthread_create() : nous utilisons ici un seul thread pour gérer tous les clients simultanément.

Le principe est le suivant : au lieu de créer un nouveau processus pour chaque client, nous surveillons tous les descripteurs de socket (celui qui écoute et ceux des clients connectés) à l'aide de la fonction select(). Cette fonction se met en attente et nous avertit dès qu'un événement se produit sur l'un d'eux – qu'il s'agisse d'une nouvelle connexion ou d'un message arrivant.

Cette approche, appelée multiplexage d'entrées-sorties, est très efficace en mémoire car elle évite de multiplier les processus ou les threads. Elle équipe des serveurs haute performance comme Nginx ou Redis, capables de gérer des milliers de connexions avec une seule unité d'exécution.

Pour notre implémentation, nous avons ajouté un timeout de 5 secondes et affiché après chaque itération le nombre de descripteurs surveillés, illustrant ainsi le fonctionnement de ce modèle mono-thread mais concurrent.

